

FTP File Reconstruction from PCAP (C++ Network Analyzer)

Overview

This project implements a C++ utility to reconstruct files transferred using FTP by analyzing raw network traffic stored in PCAP files.

The tool parses captured packets, extracts FTP control information, performs TCP reassembly, and reconstructs transferred files from network payloads.

Problem Statement

Given:

1. An FTP server
2. A file transferred using FTP
3. A PCAP file captured on the server

Design a utility that:

- Parses the PCAP
 - Extracts raw file bytes
 - Reassembles TCP streams
 - Reconstructs the original file
-

Features

- ✓ Offline PCAP analysis using libpcap
 - ✓ FTP control channel parsing
 - ✓ Passive FTP (PASV / EPSV) support
 - ✓ Active FTP (PORT / EPRT) parsing support
 - ✓ IPv4 and IPv6 support
 - ✓ TCP segment reassembly
 - ✓ Duplicate packet handling
 - ✓ Multi-session file reconstruction
 - ✓ Modular and extensible design
-

Project Structure

```
ftp-analyzer/  
|  
├── src/  
|   └── main.cpp
```

```
|
|  ├── pcap_reader.cpp / .h
|  ├── link_layer.cpp / .h
|  ├── ftp_parser.cpp / .h
|  ├── tcp_reassembly.cpp / .h
|  └── session_manager.h
|
|  ├── CMakeLists.txt
|  └── README.md
```

High-Level Architecture

The system is organized into independent modules.



Architecture Components

1. Main Module (main.cpp)

Responsible for:

- Parsing CLI arguments
- Starting PCAP processing
- Triggering TCP reassembly

2. PCAP Reader (`pcap_reader.*`)

Responsible for:

- Opening PCAP file
- Reading packets using libpcap
- Detecting link-layer offsets
- Parsing IPv4 / IPv6 headers
- Parsing TCP headers
- Extracting payload
- Identifying FTP control/data channels

3. Link Layer Handler (`link_layer.*`)

Responsible for:

- Detecting capture type (Ethernet / Loopback)
- Calculating network-layer offset

4. FTP Parser (`ftp_parser.*`)

Responsible for parsing FTP control messages.

Supported commands:

Mode	Command	Protocol
Passive	PASV	IPv4
Passive	EPSV	IPv6
Active	PORT	IPv4
Active	EPRT	IPv6

Extracts data channel ports.

5. Session Manager (`session_manager.h`)

Defines core data structures:

- TCP Segment
- Unified IPv4 / IPv6 Address
- Connection Key
- Session Map

Provides unique session identification.

6. TCP Reassembly (`tcp_reassembly.*`)

Responsible for:

- Sorting segments by sequence number
- Removing duplicates
- Writing payloads to files
- Handling multiple sessions

Data Flow



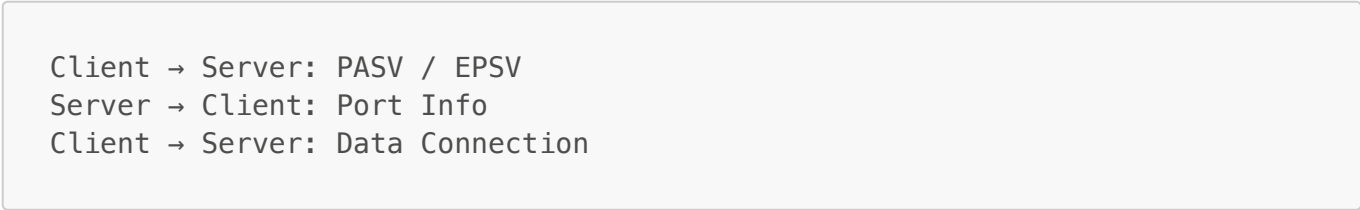
FTP Protocol Handling

FTP Connections

FTP uses two connections:

Channel	Purpose	Port
Control	Commands	21
Data	File Transfer	Dynamic

Passive Mode



Active Mode

```
Client → Server: PORT / EPRT  
Server → Client: Data Connection
```

TCP Reassembly Strategy

The TCP reassembly process follows these steps:

- Store segments per connection
- Sort segments by sequence number
- Skip duplicate packets
- Write payload data in order

This approach handles:

- ✓ Out-of-order packets
- ✓ Retransmissions
- ✓ Overlapping segments

Environment Setup

1. Install Dependencies

Dependencies

- libpcap
- CMake
- C++17

macOS

```
brew install pure-ftpd libpcap cmake  
xcode-select --install
```

Linux

```
sudo apt install libpcap-dev cmake g++
```

2. FTP Server Configuration (macOS)

Create FTP Root

```
sudo mkdir -p /tmp/ftp  
sudo chown $USER /tmp/ftp
```

Create Virtual User

```
sudo pure-pw useradd ftpuser \  
-u $USER \  
-d /tmp/ftp \  
-m
```

Start Server

```
sudo pure-ftpd -B -l puredb:/opt/homebrew/etc/pureftpd.pdb
```

Verify:

```
sudo lsof -i :21
```

Packet Capture

Start capture before FTP transfer:

```
sudo tcpdump -i any -w ftp.pcap
```

Note: -i any is required on macOS to capture loopback traffic.

FTP File Transfer

```
ftp -4 localhost  
# OR  
ftp -6 ::1 # for ipv6
```

Inside FTP:

```
binary  
passive  
put sample.pdf  
bye
```

Build Instructions

Build

```
mkdir build
cd build
cmake ..
make
```

► Usage

```
./ftp_analyzer <pcap_file> <output_prefix>
```

Example

```
./ftp_analyzer ftp.pcap recovered
```

Output

```
recovered_1
recovered_2
...
```

Testing Procedure

1. Start the FTP server
2. Capture network traffic

```
sudo tcpdump -i lo0 -w ftp.pcap
```

3. Upload file via FTP
4. Stop capture
5. Run the analyzer
6. Verify file integrity

```
shasum original.pdf recovered_1
```

Current Capabilities

Feature	Status
IPv4	✓
IPv6	✓
Passive FTP	✓
Active FTP	✓
Multi-Session	✓
TCP Reassembly	✓
Duplicate Handling	✓

Learning Outcomes

This project demonstrates:

- Low-level packet analysis
- TCP protocol understanding
- FTP protocol internals
- Network debugging
- Modular C++ design
- Systems programming

Conclusion

This project reconstructs FTP-transferred files directly from network traffic.

It demonstrates practical understanding of:

- Networking protocols
- Packet captures
- TCP reassembly
- System-level C++ development

Architecture & Design Documents

Detailed system design and performance documentation:

-  [Scalable FTP Architecture Design](#)
-  [100 Gbps Architecture Diagram \(SVG\)](#)

